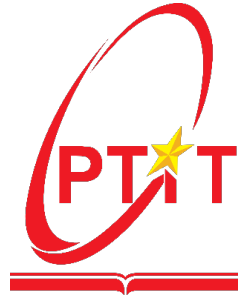


HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN I



BÁO CÁO BÀI TẬP LỚN
MÔN XỬ LÝ ẢNH

ĐỀ TÀI:
TRIỂN KHAI THUẬT TOÁN ANN CHO BÀI TOÁN
PHÂN LOẠI SỐ VIẾT TAY (0 – 9)

Nhóm lớp: 12

Nhóm bài tập lớn: 14

Giáo viên hướng dẫn: TS. Phạm Hoàng Việt

Sinh viên thực hiện:

Trần Hữu Phúc B22DCCN634

Bùi Ngọc Vũ B22DCCN910

HÀ NỘI, 2025

MỤC LỤC

A. MỞ ĐẦU	4
1. Lý do chọn đề tài	4
2. Mục tiêu nghiên cứu	5
3. Nội dung nghiên cứu.....	5
4. Đối tượng, phạm vi, phương pháp nghiên cứu	5
B. NỘI DUNG	7
CHƯƠNG 1: NGHIÊN CỨU VỀ MẠNG NƠ-RON NHÂN TẠO VÀ BÀI TOÁN PHÂN LOẠI SỐ VIẾT TAY	7
1.1. Khái niệm cơ bản về mạng nơ-ron nhân tạo	7
1.2. Bài toán phân loại số viết tay MNIST	8
1.3. Các thành phần cơ bản của mạng nơ-ron.....	9
1.3.1. Lớp tuyến tính (Linear Layer)	9
1.3.2. Hàm kích hoạt (Activation Function)	9
1.3.3. Hàm mất mát (Loss Function)	10
1.3.4. Thuật toán tối ưu hóa (Optimizer)	11
CHƯƠNG 2: TRIỂN KHAI MẠNG NƠ-RON NHÂN TẠO TỪ ĐẦU.....	12
2.1. Kiến trúc mạng nơ-ron được đề xuất	12
2.2. Triển khai các thành phần cơ bản	12
2.2.1. Lớp tuyến tính (Linear Layer)	12
2.2.2. Hàm kích hoạt ReLU	14
2.2.3. Hàm kích hoạt Softmax	15
2.2.4. Hàm mất mát Cross-Entropy	16

2.2.5. Thuật toán tối ưu hóa Adam	17
2.2.6. Lớp Flatten.....	19
2.3. Quy trình huấn luyện mô hình	20
2.4. Cài đặt và triển khai	20
2.4.1. Cấu hình môi trường phát triển.....	20
2.4.2. Cấu hình mô hình.....	20
2.4.3. Cấu hình dữ liệu.....	21
2.4.4. Cấu hình huấn luyện	22
CHƯƠNG 3: KẾT QUẢ VÀ ĐÁNH GIÁ.....	24
3.1. Kết quả huấn luyện	24
3.2. Đánh giá hiệu suất mô hình	25
3.3. Phân tích độ tin cậy và lỗi phân loại.....	28
3.4. Phân tích kết quả và so sánh	29
C. KẾT LUẬN.....	31
1. Kết luận.....	31
2. Hướng phát triển	32

DANH MỤC BẢNG BIỂU

Bảng 1: Bảng đánh giá hiệu suất mô hình theo từng lớp	27
Biểu đồ 1: Biểu đồ lịch sử huấn luyện	24
Biểu đồ 2: Biểu đồ ma trận nhầm lẫn	26
Biểu đồ 3: Biểu đồ giá trị Gradient Norm trong quá trình huấn luyện.....	27

DANH MỤC HÌNH ẢNH

Hình 1: Kiến trúc của một mạng nơ-ron nhân tạo cơ bản	7
Hình 2: Bộ dữ liệu MNIST	8

A. MỞ ĐẦU

1. Lý do chọn đề tài

Trong thời đại công nghệ số hiện nay, việc nhận dạng và phân loại chữ viết tay đóng vai trò quan trọng trong nhiều ứng dụng thực tế như nhận dạng mã bưu điện, xử lý tài liệu số hóa, và các hệ thống nhập liệu thông minh. Bài toán phân loại số viết tay là một trong những bài toán cơ bản và quan trọng nhất trong lĩnh vực thị giác máy tính và học máy, đóng vai trò nền tảng cho việc phát triển các hệ thống nhận dạng phức tạp hơn.

Mạng nơ-ron nhân tạo (Artificial Neural Network - ANN) đã chứng minh được hiệu quả vượt trội trong việc giải quyết các bài toán phân loại hình ảnh. Tuy nhiên, việc sử dụng các thư viện học sâu hiện đại như PyTorch hay TensorFlow thường che giấu các chi tiết quan trọng về cách thức hoạt động bên trong của mạng nơ-ron. Việc triển khai mạng nơ-ron từ đầu, không sử dụng các module có sẵn, sẽ giúp hiểu sâu hơn về nguyên lý hoạt động của các thành phần cơ bản như lan truyền thuận (forward propagation), lan truyền ngược (backward propagation), và các thuật toán tối ưu hóa.

Bộ dữ liệu MNIST (Modified National Institute of Standards and Technology) là một bộ dữ liệu chuẩn trong cộng đồng nghiên cứu học máy, bao gồm 70.000 hình ảnh số viết tay từ 0 đến 9. Đây là một bộ dữ liệu lý tưởng để nghiên cứu và phát triển các thuật toán phân loại do tính đơn giản về kích thước và độ phức tạp vừa phải, đồng thời vẫn đảm bảo tính thách thức đủ để đánh giá hiệu quả của các phương pháp.

Từ những lý do trên, chúng tôi quyết định chọn đề tài "Triển khai thuật toán ANN cho bài toán phân loại số viết tay" nhằm mục đích xây dựng một hệ thống phân loại số viết tay hoàn chỉnh từ đầu, không sử dụng các module có sẵn của các framework học sâu, qua đó nâng cao hiểu biết về cơ chế hoạt động của mạng nơ-ron nhân tạo và khả năng áp dụng vào các bài toán thực tế.

2. Mục tiêu nghiên cứu

Nghiên cứu và triển khai từ đầu các thành phần cơ bản của mạng nơ-ron nhân tạo bao gồm lớp tuyến tính, hàm kích hoạt, hàm mất mát và thuật toán tối ưu hóa.

Xây dựng một mô hình mạng nơ-ron hoàn chỉnh để giải quyết bài toán phân loại số viết tay trên bộ dữ liệu MNIST, đạt được độ chính xác cao và ổn định.

Đánh giá và phân tích hiệu suất của mô hình thông qua các chỉ số như độ chính xác, ma trận nhầm lẫn, và các biểu đồ trực quan hóa quá trình huấn luyện.

3. Nội dung nghiên cứu

- Nội dung 1: Nghiên cứu về mạng nơ-ron nhân tạo và bài toán phân loại số viết tay, bao gồm các khái niệm cơ bản về ANN, đặc điểm của bộ dữ liệu MNIST, và các thành phần cơ bản của mạng nơ-ron.
- Nội dung 2: Triển khai từ đầu các thành phần của mạng nơ-ron nhân tạo, bao gồm lớp tuyến tính với lan truyền thuận và lan truyền ngược, các hàm kích hoạt ReLU và Softmax, hàm mất mát Cross-Entropy, và thuật toán tối ưu hóa Adam.
- Nội dung 3: Huấn luyện mô hình trên bộ dữ liệu MNIST, đánh giá hiệu suất, và phân tích kết quả thông qua các biểu đồ và chỉ số đánh giá.

4. Đối tượng, phạm vi, phương pháp nghiên cứu

Đối tượng nghiên cứu:

- Mạng nơ-ron nhân tạo (Artificial Neural Network)
- Bài toán phân loại số viết tay
- Bộ dữ liệu MNIST
- Các thuật toán lan truyền thuận và lan truyền ngược
- Thuật toán tối ưu hóa Adam

Phạm vi nghiên cứu:

- Triển khai mạng nơ-ron đa lớp (Multi-layer Perceptron) từ đầu
- Phân loại 10 lớp số viết tay (0-9) trên bộ dữ liệu MNIST
- Sử dụng ngôn ngữ lập trình Python và thư viện PyTorch cho các phép toán tensor

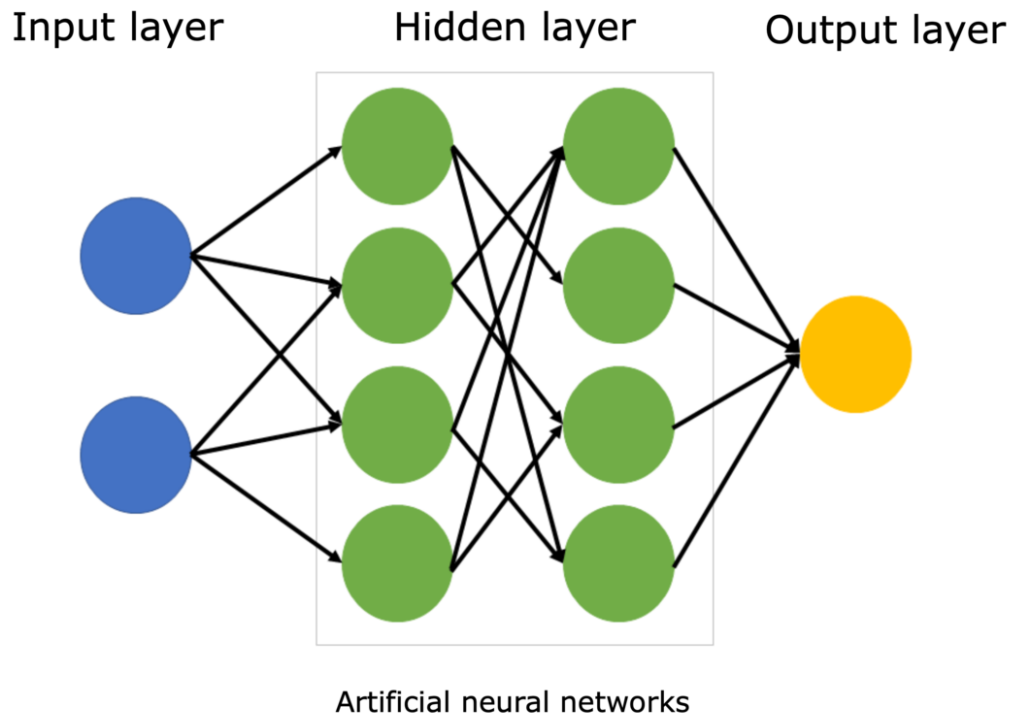
Phương pháp nghiên cứu:

- Phương pháp nghiên cứu mô hình hóa: Xây dựng mô hình mạng nơ-ron với kiến trúc phù hợp cho bài toán phân loại, thiết kế các thành phần cơ bản và quy trình huấn luyện.
- Phương pháp nghiên cứu thực nghiệm: Cài đặt và triển khai mô hình trên bộ dữ liệu thực tế, thực hiện quá trình huấn luyện và đánh giá hiệu suất.
- Phương pháp nghiên cứu tham khảo: Tổng hợp và phân tích các nghiên cứu liên quan về mạng nơ-ron nhân tạo và bài toán phân loại số viết tay, đánh giá ưu nhược điểm của các phương pháp hiện có.
- Phương pháp phân tích và đánh giá: Sử dụng các chỉ số đánh giá như độ chính xác, ma trận nhầm lẫn, và các biểu đồ trực quan để phân tích kết quả và đưa ra nhận xét.

B. NỘI DUNG

CHƯƠNG 1: NGHIÊN CỨU VỀ MẠNG NƠ-RON NHÂN TẠO VÀ BÀI TOÁN PHÂN LOẠI SỐ VIẾT TAY

1.1. Khái niệm cơ bản về mạng nơ-ron nhân tạo



Hình 1: Kiến trúc của một mạng nơ-ron nhân tạo cơ bản

Mạng nơ-ron nhân tạo (Artificial Neural Network - ANN) là một mô hình tính toán được lấy cảm hứng từ cấu trúc và chức năng của hệ thần kinh sinh học. Mạng nơ-ron nhân tạo bao gồm các đơn vị xử lý được gọi là nơ-ron (neurons) hoặc nút (nodes), được kết nối với nhau thông qua các trọng số (weights). Mỗi nơ-ron nhận đầu vào từ các nơ-ron trước đó, thực hiện một phép tính tổng có trọng số, sau đó áp dụng một hàm kích hoạt để tạo ra đầu ra.

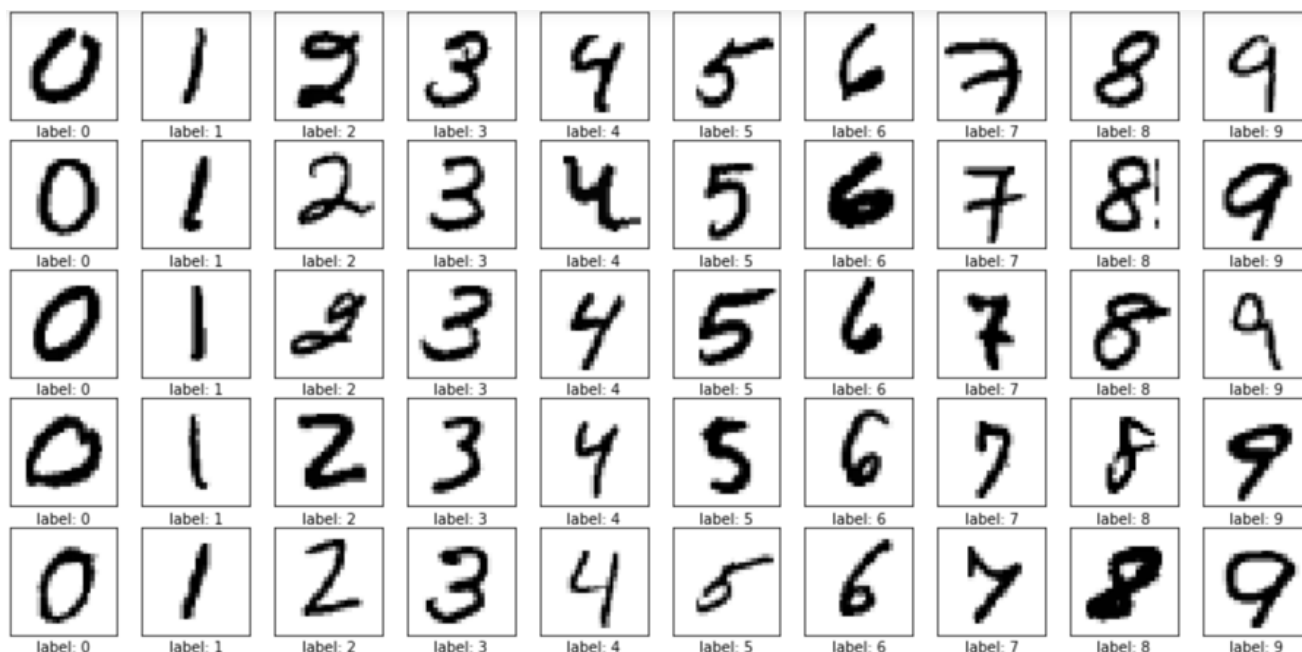
Mạng nơ-ron đa lớp (Multi-layer Perceptron - MLP) là một kiến trúc phổ biến của ANN, bao gồm một lớp đầu vào (input layer), một hoặc nhiều lớp ẩn (hidden layers), và một lớp đầu ra (output layer). Các lớp được kết nối đầy đủ

(fully connected), nghĩa là mỗi nơ-ron trong một lớp được kết nối với tất cả các nơ-ron trong lớp tiếp theo.

Quá trình học của mạng nơ-ron được thực hiện thông qua hai giai đoạn chính: lan truyền thuận (forward propagation) và lan truyền ngược (backward propagation). Trong giai đoạn lan truyền thuận, dữ liệu đầu vào được truyền qua các lớp của mạng để tạo ra dự đoán. Trong giai đoạn lan truyền ngược, gradient của hàm mất mát được tính toán và truyền ngược lại qua mạng để cập nhật các trọng số, giúp mạng học từ các lỗi dự đoán.

1.2. Bài toán phân loại số viết tay MNIST

Bộ dữ liệu MNIST là một bộ dữ liệu chuẩn trong cộng đồng nghiên cứu học máy, được tạo ra bởi Viện Tiêu chuẩn và Công nghệ Quốc gia Hoa Kỳ (NIST). Bộ dữ liệu này bao gồm 70.000 hình ảnh số viết tay từ 0 đến 9, được chia thành 60.000 hình ảnh cho tập huấn luyện và 10.000 hình ảnh cho tập kiểm tra.



Hình 2: Bộ dữ liệu MNIST

Mỗi hình ảnh trong bộ dữ liệu MNIST có kích thước 28×28 pixel, được biểu diễn dưới dạng ảnh xám với giá trị pixel từ 0 đến 255. Các hình ảnh được

chuẩn hóa và tiền xử lý để phù hợp với đầu vào của mạng nơ-ron. Bài toán phân loại số viết tay là một bài toán phân loại đa lớp (multi-class classification), trong đó mô hình cần học cách phân loại mỗi hình ảnh đầu vào vào một trong 10 lớp tương ứng với các chữ số từ 0 đến 9.

Độ phức tạp của bài toán nằm ở sự đa dạng trong cách viết tay của con người. Cùng một chữ số có thể được viết theo nhiều cách khác nhau về kích thước, độ nghiêng, độ dày nét, và vị trí trong khung hình. Điều này đòi hỏi mô hình phải có khả năng học được các đặc trưng tổng quát để nhận diện chữ số một cách chính xác bất kể sự biến đổi này.

1.3. Các thành phần cơ bản của mạng nơ-ron

1.3.1. Lớp tuyến tính (Linear Layer)

Lớp tuyến tính, còn được gọi là lớp fully connected hoặc dense layer, là thành phần cơ bản nhất của mạng nơ-ron. Lớp này thực hiện phép biến đổi tuyến tính trên đầu vào bằng cách nhân ma trận đầu vào với ma trận trọng số và cộng với vector bias. Công thức toán học của lớp tuyến tính được biểu diễn như sau:

$$y = xW^T + b$$

Trong đó x là vector đầu vào, W là ma trận trọng số, b là vector bias, và y là vector đầu ra. Trong quá trình lan truyền ngược, gradient của hàm mất mát được tính toán đối với các trọng số và bias để cập nhật chúng thông qua thuật toán tối ưu hóa.

1.3.2. Hàm kích hoạt (Activation Function)

Hàm kích hoạt được sử dụng để đưa tính phi tuyến vào mạng nơ-ron, cho phép mạng học được các mối quan hệ phức tạp trong dữ liệu. Có nhiều loại hàm kích hoạt khác nhau, mỗi loại có những đặc điểm và ứng dụng riêng.

Hàm ReLU (Rectified Linear Unit) là một trong những hàm kích hoạt phổ biến nhất hiện nay, được định nghĩa là $f(x) = \max(0, x)$. Hàm ReLU có ưu điểm

là tính toán đơn giản và hiệu quả, đồng thời giúp giải quyết vấn đề vanishing gradient trong các mạng sâu. Tuy nhiên, ReLU có nhược điểm là có thể gây ra hiện tượng "dying ReLU" khi một số nơ-ron không bao giờ được kích hoạt.

Hàm Softmax thường được sử dụng ở lớp đầu ra của mạng nơ-ron cho bài toán phân loại đa lớp. Hàm Softmax chuyển đổi một vector các giá trị thực thành một vector xác suất, trong đó tổng các phần tử bằng 1. Công thức của hàm Softmax được biểu diễn như sau:

$$\text{Softmax}(x_i) = \frac{e^{x_i}}{\sum_{j=1}^n e^{x_j}}$$

Trong đó:

- x_i : giá trị đầu vào (logit) của lớp thứ i .
- n : tổng số lớp (classes).
- e^{x_i} : chuyển đổi giá trị x_i sang dạng mũ dương.
- Mẫu số $\sum_{j=1}^n e^{x_j}$: tổng của tất cả các giá trị mũ — giúp chuẩn hóa kết quả thành xác suất.

1.3.3. Hàm mất mát (Loss Function)

Hàm mất mát đo lường sự khác biệt giữa dự đoán của mô hình và giá trị thực tế. Đối với bài toán phân loại đa lớp, hàm mất mát Cross-Entropy là lựa chọn phổ biến nhất. Hàm Cross-Entropy được định nghĩa như sau:

$$L = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{i,c} \log(\hat{y}_{i,c})$$

Trong đó:

- L : giá trị hàm mất mát trung bình (loss) trên toàn bộ batch.
- N : số lượng mẫu (samples) trong batch.

- C : số lượng lớp (classes).
- $y_{i,c}$: giá trị nhãn thực tế của mẫu thứ i ở lớp c .
- $\hat{y}_{i,c}$: xác suất dự đoán mô hình cho mẫu i thuộc lớp c .

1.3.4. Thuật toán tối ưu hóa (Optimizer)

Thuật toán tối ưu hóa được sử dụng để cập nhật các trọng số của mạng nơ-ron dựa trên gradient của hàm mất mát. Adam (Adaptive Moment Estimation) là một trong những thuật toán tối ưu hóa phổ biến nhất hiện nay, kết hợp ưu điểm của cả thuật toán Momentum và RMSprop.

Adam duy trì hai moment estimates: moment bậc nhất (mean) và moment bậc hai (uncentered variance) của gradient. Các moment này được cập nhật theo công thức:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

Sau đó, các moment được điều chỉnh bias và trọng số được cập nhật:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t$$

Trong đó α là learning rate, β_1 và β_2 là các hệ số decay, và ϵ là một giá trị nhỏ để tránh chia cho 0.

CHƯƠNG 2: TRIỂN KHAI MẠNG NƠ-RON NHÂN TẠO TỪ ĐẦU

2.1. Kiến trúc mạng nơ-ron được đề xuất

Trong nghiên cứu này, chúng tôi đề xuất một kiến trúc mạng nơ-ron đa lớp với cấu trúc như sau: lớp đầu vào nhận 784 đặc trưng (tương ứng với 28×28 pixel của hình ảnh MNIST sau khi được làm phẳng), lớp ẩn thứ nhất gồm 256 nơ-ron với hàm kích hoạt ReLU, lớp ẩn thứ hai gồm 128 nơ-ron với hàm kích hoạt ReLU, và lớp đầu ra gồm 10 nơ-ron với hàm kích hoạt Softmax tương ứng với 10 lớp số từ 0 đến 9.

Kiến trúc này được thiết kế dựa trên sự cân bằng giữa độ phức tạp của mô hình và khả năng học các đặc trưng từ dữ liệu. Lớp ẩn thứ nhất với 256 nơ-ron có khả năng học được các đặc trưng cấp thấp từ dữ liệu đầu vào, trong khi lớp ẩn thứ hai với 128 nơ-ron giúp tổng hợp và trừu tượng hóa các đặc trưng này để đưa ra quyết định phân loại cuối cùng.

Tổng số tham số của mô hình là 235.146 tham số, bao gồm 200.960 tham số cho lớp đầu tiên (256×784 trọng số + 256 bias), 32.896 tham số cho lớp thứ hai (128×256 trọng số + 128 bias), và 1.290 tham số cho lớp đầu ra (10×128 trọng số + 10 bias). Số lượng tham số này phù hợp với độ phức tạp của bài toán và đảm bảo khả năng học hiệu quả mà không gây ra hiện tượng overfitting quá mức.

2.2. Triển khai các thành phần cơ bản

2.2.1. Lớp tuyến tính (Linear Layer)

Lớp tuyến tính là thành phần cơ bản nhất của mạng nơ-ron, thực hiện phép biến đổi tuyến tính trên dữ liệu đầu vào. Về mặt toán học, lớp tuyến tính thực hiện phép toán:

$$y = xW^T + b$$

Trong đó $x \in \mathbb{R}^{batch \times in_features}$ là ma trận đầu vào ($batch_size \times$ số đặc trưng đầu vào), $W \in \mathbb{R}^{out_features \times in_features}$ là ma trận trọng số, $b \in \mathbb{R}^{out_features}$ là vector bias, và $y \in \mathbb{R}^{batch \times out_features}$ là ma trận đầu ra.

Trong triển khai, phương thức forward thực hiện các bước sau: đầu tiên, nếu đầu vào có chiều lớn hơn 2 (ví dụ tensor 4D cho hình ảnh), nó được làm phẳng thành ma trận 2D với kích thước ($batch_size$, số đặc trưng). Sau đó, phép nhân ma trận được thực hiện giữa đầu vào và ma trận trọng số đã được chuyển vị, cuối cùng vector bias được cộng vào nếu được sử dụng. Đầu vào được lưu vào cache để sử dụng trong quá trình lan truyền ngược.

Khởi tạo trọng số đóng vai trò quan trọng trong việc đảm bảo mạng nơ-ron học hiệu quả. Trong triển khai này, phương pháp He initialization (một biến thể của Xavier initialization cho ReLU) được sử dụng với công thức:

$$W_{ij} \sim \mathcal{N}\left(0, \sqrt{\frac{2}{in_features}}\right)$$

Phương pháp này đảm bảo phương sai của đầu ra được duy trì qua các lớp khi sử dụng hàm kích hoạt ReLU, giúp tránh hiện tượng vanishing gradient và exploding gradient.

Trong quá trình lan truyền ngược, gradient của hàm mất mát cần được tính toán đối với trọng số, bias và đầu vào. Công thức tính gradient được suy ra từ quy tắc chuỗi (chain rule):

$$\frac{\partial L}{\partial W} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial W} = (\nabla_y L)^T \cdot x$$

$$\frac{\partial L}{\partial b} = \sum_{i=1}^{batch} \frac{\partial L}{\partial y_i}$$

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial x} = \nabla_y L \cdot W$$

Trong triển khai, gradient đối với trọng số được tính bằng cách nhân ma trận chuyển vị của gradient đầu ra với đầu vào đã được cache: $\nabla W = (\text{grad_output})^T \cdot \text{input}$. Gradient đối với bias được tính bằng cách lấy tổng gradient đầu ra theo chiều batch: $\text{bias_grad} = \sum(\text{grad_output}, \text{dim} = 0)$. Gradient đầu vào được tính để truyền ngược lại lớp trước: $\text{grad_input} = \text{grad_output} @ W$.

2.2.2. Hàm kích hoạt ReLU

Hàm ReLU (Rectified Linear Unit) là một trong những hàm kích hoạt phổ biến nhất trong học sâu do tính đơn giản và hiệu quả của nó. Về mặt toán học, hàm ReLU được định nghĩa như sau:

$$\text{ReLU}(x) = \max(0, x) = \begin{cases} x & \text{nếu } x > 0 \\ 0 & \text{nếu } x \leq 0 \end{cases}$$

Hàm ReLU có đạo hàm đơn giản:

$$\frac{d}{dx} \text{ReLU}(x) = \begin{cases} 1 & \text{nếu } x > 0 \\ 0 & \text{nếu } x \leq 0 \end{cases}$$

Trong triển khai, phương thức forward sử dụng hàm `torch.clamp(x, min = 0)` để áp dụng hàm ReLU, tương đương với việc giữ nguyên các giá trị dương và đặt các giá trị âm thành 0. Đầu vào được lưu vào cache để sử dụng trong quá trình tính gradient.

Trong phương thức backward, gradient chỉ được truyền qua các nơ-ron có giá trị đầu vào lớn hơn 0. Cụ thể, một mask được tạo ra dựa trên đầu vào đã được cache: `relu_mask = (input_cache > 0).float()`, sau đó gradient đầu ra được nhân với mask này: `grad_input = grad_output * relu_mask`. Điều này có nghĩa là các nơ-ron không được kích hoạt (có giá trị đầu vào ≤ 0) sẽ không nhận được gradient, dẫn đến trọng số của chúng không được cập nhật trong bước đó.

Ưu điểm chính của ReLU là tính toán nhanh và giúp giải quyết vấn đề vanishing gradient trong các mạng sâu. Tuy nhiên, ReLU có nhược điểm là có thể gây ra hiện tượng "dying ReLU" khi một số nơ-ron không bao giờ được kích hoạt sau một số bước cập nhật, dẫn đến việc chúng trở nên "chết" và không đóng góp vào quá trình học.

2.2.3. Hàm kích hoạt Softmax

Hàm Softmax được sử dụng ở lớp đầu ra của mạng nơ-ron cho bài toán phân loại đa lớp. Hàm này chuyển đổi một vector các giá trị thực (logits) thành một vector xác suất, trong đó tổng các phần tử bằng 1. Công thức toán học của hàm Softmax được định nghĩa như sau:

$$\text{Softmax}(x_i) = \frac{e^{x_i}}{\sum_{j=1}^n e^{x_j}}$$

Trong đó x_i là phần tử thứ i của vector đầu vào, và n là số lượng phần tử trong vector.

Một vấn đề quan trọng khi triển khai hàm Softmax là tính ổn định số học. Khi các giá trị x_i lớn, việc tính e^{x_i} có thể gây ra hiện tượng overflow (tràn số). Để giải quyết vấn đề này, một kỹ thuật được sử dụng là trừ đi giá trị lớn nhất trong vector trước khi áp dụng hàm exponential:

$$\text{Softmax}(x_i) = \frac{e^{x_i - \max(x)}}{\sum_{j=1}^n e^{x_j - \max(x)}}$$

Việc trừ đi $\max(x)$ không thay đổi kết quả của hàm Softmax do tính chất của hàm exponential, nhưng đảm bảo rằng giá trị lớn nhất sau khi trừ sẽ là 0, và tất cả các giá trị khác sẽ là số âm, do đó $e^{x_i - \max(x)} \leq 1$, tránh được hiện tượng overflow.

Trong triển khai, phương thức forward thực hiện các bước sau: tìm giá trị lớn nhất trong vector đầu vào theo chiều được chỉ định (thường là chiều cuối cùng), trừ giá trị này khỏi tất cả các phần tử, áp dụng hàm exponential, và cuối

cùng chuẩn hóa bằng cách chia cho tổng. Kết quả được lưu vào cache để sử dụng trong quá trình tính gradient.

Đạo hàm của hàm Softmax có dạng đặc biệt. Nếu $y_i = \text{Softmax}(x_i)$, thì:

$$\frac{\partial y_i}{\partial x_j} = \begin{cases} y_i(1 - y_i) & \text{nếu } i = j \\ -y_i y_j & \text{nếu } i \neq j \end{cases}$$

Công thức tổng quát có thể được viết dưới dạng:

$$\frac{\partial y_i}{\partial x_j} = y_i(\delta_{ij} - y_j)$$

Trong đó δ_{ij} là delta Kronecker. Trong phương thức backward, gradient được tính dựa trên công thức này, sử dụng kết quả đã được cache từ forward pass để tăng hiệu quả tính toán.

2.2.4. Hàm mất mát Cross-Entropy

Hàm mất mát Cross-Entropy là lựa chọn tiêu chuẩn cho bài toán phân loại đa lớp. Về mặt toán học, hàm Cross-Entropy đo lường sự khác biệt giữa phân phối xác suất thực tế và phân phối xác suất dự đoán. Công thức của hàm Cross-Entropy được định nghĩa như sau:

$$L = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{i,c} \log(\hat{y}_{i,c})$$

Trong đó:

- N là số lượng mẫu trong batch.
- C là số lượng lớp.
- $y_{i,c}$ là nhãn thực tế dưới dạng one-hot encoding (1 nếu mẫu i thuộc lớp c , 0 nếu không).
- $\hat{y}_{i,c}$ là xác suất dự đoán của lớp c cho mẫu i .

Trong triển khai, hàm Cross-Entropy được kết hợp với hàm Softmax để tăng hiệu quả tính toán và ổn định số học. Thay vì tính Softmax riêng biệt rồi mới tính Cross-Entropy, việc kết hợp hai hàm này cho phép tính toán gradient một cách trực tiếp và hiệu quả hơn.

Trong phương thức forward, đầu vào logits được chuyển đổi thành xác suất thông qua hàm Softmax. Sau đó, nhãn thực tế được chuyển đổi sang dạng one-hot encoding nếu chưa ở dạng này. Để tránh hiện tượng $\log(0)$ (gây ra giá trị vô cực), các giá trị xác suất được giới hạn trong khoảng $[\epsilon, 1 - \epsilon]$ với $\epsilon = 10^{-15}$ bằng cách sử dụng hàm `torch.clamp`. Cuối cùng, hàm mất mát được tính bằng công thức Cross-Entropy và được chuẩn hóa bằng cách chia cho kích thước batch.

Đạo hàm của hàm Cross-Entropy kết hợp với Softmax có dạng đơn giản và hiệu quả:

$$\frac{\partial L}{\partial x_i} = \hat{y}_i - y_i$$

Trong đó:

- x_i là logit thứ i .
- $\hat{y}_i$ là xác suất dự đoán (sau Softmax).
- y_i là nhãn thực tế.

Công thức này cho thấy gradient đơn giản là sự khác biệt giữa xác suất dự đoán và nhãn thực tế. Trong phương thức backward, gradient được tính trực tiếp từ công thức này, sử dụng kết quả Softmax đã được cache và nhãn thực tế đã được chuyển đổi sang one-hot encoding.

2.2.5. Thuật toán tối ưu hóa Adam

Adam (Adaptive Moment Estimation) là một thuật toán tối ưu hóa stochastic gradient descent được đề xuất bởi Kingma và Ba vào năm 2014. Adam kết hợp ưu điểm của cả thuật toán Momentum (sử dụng moment bậc nhất) và

RMSprop (sử dụng moment bậc hai), đồng thời thêm cơ chế bias correction để cải thiện hiệu suất trong giai đoạn đầu của quá trình huấn luyện.

Thuật toán Adam duy trì hai moment estimates cho mỗi tham số: moment bậc nhất m_t (ước lượng trung bình của gradient) và moment bậc hai v_t (ước lượng phương sai không tâm của gradient). Các moment này được cập nhật theo công thức:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

Trong đó:

- g_t là gradient tại bước thời gian t .
- β_1 và β_2 là các hệ số decay cho moment bậc nhất và bậc hai, thường được đặt là 0.9 và 0.999 tương ứng.

Do các moment được khởi tạo bằng 0, chúng có xu hướng bị thiên lệch về 0 trong giai đoạn đầu, đặc biệt khi t nhỏ. Để khắc phục vấn đề này, Adam sử dụng bias correction:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

Các giá trị đã được điều chỉnh bias này sau đó được sử dụng để cập nhật tham số:

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t$$

Trong đó:

- α là learning rate (tốc độ học), thường được đặt là 0.001.
- ϵ là một giá trị nhỏ (thường là 10^{-8}) để tránh chia cho 0.

Trong triển khai, mỗi tham số có một cặp moment estimates riêng được khởi tạo bằng 0. Trong mỗi bước cập nhật (phương thức step), bước thời gian t được tăng lên 1, và với mỗi tham số và gradient tương ứng, các moment được cập nhật theo công thức trên. Sau đó, bias correction được áp dụng, và tham số được cập nhật sử dụng learning rate adaptive được tính từ các moment đã được điều chỉnh.

Ưu điểm chính của Adam là khả năng tự động điều chỉnh learning rate cho từng tham số dựa trên lịch sử gradient, giúp quá trình huấn luyện ổn định và hội tụ nhanh hơn so với các thuật toán tối ưu hóa truyền thống như SGD. Adam đặc biệt hiệu quả khi làm việc với các mạng nơ-ron sâu và các bài toán có dữ liệu thưa thớt hoặc nhiễu.

2.2.6. Lớp Flatten

Lớp Flatten không phải là một lớp tính toán phức tạp, nhưng đóng vai trò quan trọng trong việc chuyển đổi dữ liệu từ dạng đa chiều sang dạng 2D để phù hợp với lớp tuyến tính. Trong bài toán phân loại hình ảnh, đầu vào thường là tensor 4D với kích thước (batch_size, channels, height, width), trong khi lớp tuyến tính yêu cầu đầu vào là ma trận 2D với kích thước (batch_size, số đặc trưng).

Lớp Flatten thực hiện phép biến đổi đơn giản là làm phẳng (flatten) các chiều không phải batch thành một chiều duy nhất. Ví dụ, một tensor với kích thước (batch_size, 1, 28, 28) sẽ được chuyển đổi thành (batch_size, 784) sau khi đi qua lớp Flatten.

Trong phương thức forward, hình dạng đầu vào được lưu vào cache để sử dụng trong quá trình lan truyền ngược. Đầu vào được làm phẳng bằng cách sử dụng phương thức view hoặc flatten của PyTorch, giữ nguyên chiều batch và làm phẳng tất cả các chiều còn lại.

Trong phương thức backward, gradient đầu ra (đã được làm phẳng) được chuyển đổi lại về hình dạng ban đầu của đầu vào bằng cách sử dụng phương thức

view với hình dạng đã được cache. Điều này đảm bảo gradient có cùng hình dạng với đầu vào ban đầu, cho phép lan truyền ngược hoạt động chính xác.

2.3. Quy trình huấn luyện mô hình

Quy trình huấn luyện mô hình được thực hiện theo các bước sau. Đầu tiên, bộ dữ liệu MNIST được tải về và chia thành ba tập: tập huấn luyện (48.000 mẫu, 80%), tập validation (12.000 mẫu, 20%), và tập kiểm tra (10.000 mẫu). Dữ liệu được tiền xử lý bằng cách resize về kích thước 28×28 , chuyển đổi thành tensor, và chuẩn hóa về khoảng $[-1, 1]$.

2.4. Cài đặt và triển khai

2.4.1. Cấu hình môi trường phát triển

Hệ thống được triển khai bằng ngôn ngữ lập trình Python 3.10.15, sử dụng thư viện PyTorch 2.7.0 cho các phép toán tensor và xử lý dữ liệu. Các thư viện hỗ trợ khác bao gồm NumPy 1.26.4 cho tính toán số học, Matplotlib 3.9.2 và Seaborn cho trực quan hóa dữ liệu, và scikit-learn cho các chỉ số đánh giá như confusion matrix và classification report.

Môi trường phát triển được thiết lập trên hệ điều hành Windows 11 với Anaconda 9.0. Toàn bộ mã nguồn được tổ chức trong một module duy nhất nn.py chứa tất cả các lớp và hàm cần thiết, cùng với một Jupyter notebook train_model.ipynb chứa quy trình huấn luyện và đánh giá mô hình. Các thư mục model_checkpoints và training_plots được tạo tự động để lưu trữ các checkpoint và biểu đồ trực quan hóa.

2.4.2. Cấu hình mô hình

Kiến trúc mạng nơ-ron được cấu hình với các tham số sau:

- Kích thước đầu vào: 784 đặc trưng (tương ứng với 28×28 pixel của hình ảnh MNIST sau khi được làm phẳng)

- Lớp ẩn thứ nhất: 256 nơ-ron với hàm kích hoạt ReLU
- Lớp ẩn thứ hai: 128 nơ-ron với hàm kích hoạt ReLU
- Lớp đầu ra: 10 nơ-ron với hàm kích hoạt Softmax (tương ứng với 10 lớp số từ 0 đến 9)

Tổng số tham số của mô hình là 235.146 tham số, được phân bổ như sau: 200.960 tham số cho lớp đầu tiên (256×784 trọng số + 256 bias), 32.896 tham số cho lớp thứ hai (128×256 trọng số + 128 bias), và 1.290 tham số cho lớp đầu ra (10×128 trọng số + 10 bias).

2.4.3. Cấu hình dữ liệu

Bộ dữ liệu MNIST được tải về tự động thông qua thư viện *torchvision.datasets.MNIST*. Dữ liệu được chia thành ba tập với tỷ lệ như sau:

- Tập huấn luyện: 48.000 mẫu (80% của 60.000 mẫu huấn luyện ban đầu)
- Tập validation: 12.000 mẫu (20% của 60.000 mẫu huấn luyện ban đầu)
- Tập kiểm tra: 10.000 mẫu (tập kiểm tra riêng biệt của MNIST)

Việc chia dữ liệu được thực hiện với random seed cố định (42) để đảm bảo tính tái lập của kết quả. Tập validation được sử dụng để theo dõi hiệu suất trong quá trình huấn luyện và quyết định khi nào dừng huấn luyện, trong khi tập kiểm tra chỉ được sử dụng một lần duy nhất ở cuối để đánh giá hiệu suất cuối cùng của mô hình.

Dữ liệu được tiền xử lý thông qua một pipeline biến đổi bao gồm:

- Resize: Đảm bảo tất cả hình ảnh có kích thước 28×28 pixel
- ToTensor: Chuyển đổi hình ảnh từ định dạng PIL Image sang tensor PyTorch và chuẩn hóa giá trị pixel về khoảng $[0, 1]$
- Normalize: Chuẩn hóa giá trị pixel về khoảng $[-1, 1]$ bằng công thức
$$\frac{(pixel - 0.5)}{0.5}$$

2.4.4. Cấu hình huấn luyện

Quá trình huấn luyện được cấu hình với các tham số sau:

Thuật toán tối ưu hóa: Adam optimizer với các tham số:

- Learning rate: 0.001 - được chọn dựa trên thực nghiệm, đảm bảo quá trình học ổn định và hội tụ tốt
- Beta1: 0.9 - hệ số decay cho moment bậc nhất, giúp làm mịn gradient
- Beta2: 0.999 - hệ số decay cho moment bậc hai, giúp điều chỉnh learning rate adaptive
- Epsilon: $1e-8$ - giá trị nhỏ để tránh chia cho 0 trong công thức cập nhật

Hàm mất mát: Cross-Entropy Loss kết hợp với Softmax, phù hợp cho bài toán phân loại đa lớp.

Batch size: 64 - được chọn để cân bằng giữa tốc độ huấn luyện và độ ổn định của gradient. Batch size này phù hợp với kích thước bộ dữ liệu và tài nguyên tính toán có sẵn.

Số epoch tối đa: 100 - mặc dù quá trình huấn luyện thường dừng sớm do early stopping, giới hạn này đảm bảo mô hình có đủ thời gian để hội tụ nếu cần.

Early stopping: Được cấu hình với hai tham số:

- Patience: 10 epoch - số epoch tối đa mà mô hình có thể không cải thiện trước khi dừng huấn luyện. Giá trị này được chọn để cho phép mô hình vượt qua các giai đoạn dao động nhỏ trong quá trình học.
- Min delta: 0.001 - mức cải thiện tối thiểu (0.1%) để được coi là một cải thiện có ý nghĩa. Điều này giúp tránh việc lưu checkpoint cho những cải thiện không đáng kể.

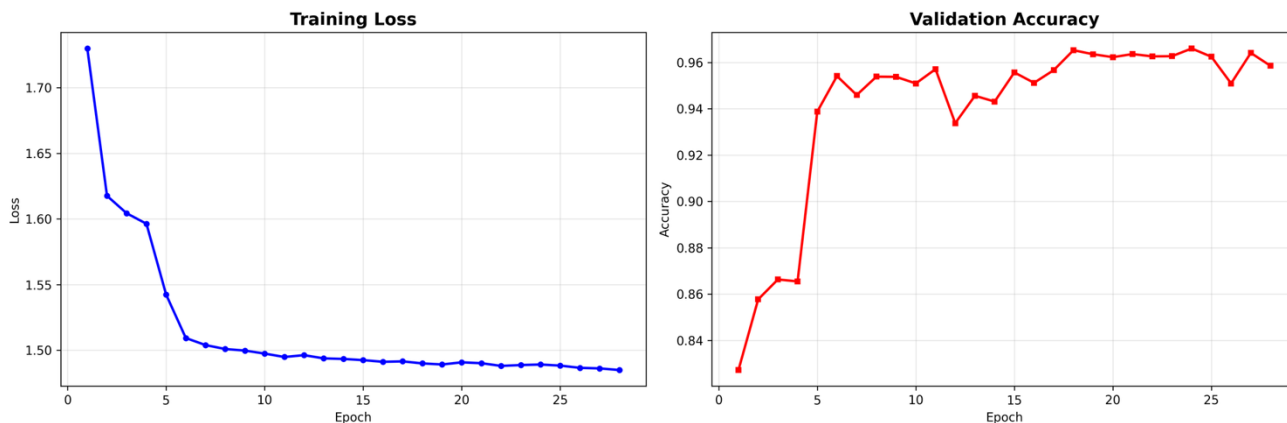
Quy trình huấn luyện: Trong mỗi epoch, mô hình được huấn luyện trên toàn bộ tập huấn luyện với các batch được shuffle ngẫu nhiên. Sau mỗi epoch, mô hình

được đánh giá trên tập validation. Nếu độ chính xác validation cải thiện với mức tối thiểu là min_delta , mô hình được lưu lại như một checkpoint. Quá trình huấn luyện dừng lại nếu không có cải thiện trong patience epoch liên tiếp.

CHƯƠNG 3: KẾT QUẢ VÀ ĐÁNH GIÁ

3.1. Kết quả huấn luyện

Biểu đồ 1: Biểu đồ lịch sử huấn luyện



Quá trình huấn luyện mô hình được thực hiện với cấu hình tối đa 100 epoch, nhưng đã dừng sớm ở epoch thứ 28 do không có cải thiện trong độ chính xác validation trong 10 epoch liên tiếp. Thời gian huấn luyện tổng cộng là 281.53 giây, tương đương khoảng 4.7 phút, cho thấy hiệu suất tính toán tốt của mô hình.

Độ chính xác trên tập validation đạt giá trị tốt nhất là 96.53% ở epoch thứ 18. Độ chính xác trên tập kiểm tra đạt 96.30%, chỉ thấp hơn 0.23% so với độ chính xác validation tốt nhất, cho thấy mô hình có khả năng tổng quát hóa tốt và không bị overfitting đáng kể. Sự chênh lệch nhỏ này là bình thường và chấp nhận được trong các bài toán học máy.

Quá trình huấn luyện cho thấy sự cải thiện ổn định và liên tục qua các epoch. Ở epoch đầu tiên, độ chính xác validation đạt 82.73% với hàm mất mát huấn luyện là 1.73. Sau đó, mô hình cải thiện nhanh chóng: đạt 85.77% ở epoch thứ 2, 86.63% ở epoch thứ 3. Một bước nhảy vọt đáng kể xảy ra ở epoch thứ 5, khi độ chính xác tăng từ khoảng 86.63% lên 93.88%, tương ứng với việc hàm mất mát giảm từ 1.60 xuống 1.54. Sự cải thiện đáng kể tiếp theo xảy ra từ epoch thứ 5 đến epoch thứ 6, khi độ chính xác tăng từ 93.88% lên 95.41%, với hàm mất mát giảm từ 1.54 xuống 1.51.

Sau epoch thứ 6, mô hình tiếp tục cải thiện dần dần với các bước nhỏ hơn. Ở epoch thứ 11, độ chính xác đạt 95.71%, và cuối cùng đạt đỉnh ở epoch thứ 18 với 96.53%. Từ epoch thứ 18 trở đi, mô hình không còn cải thiện thêm, và sau 10 epoch không có cải thiện, quá trình huấn luyện được dừng lại.

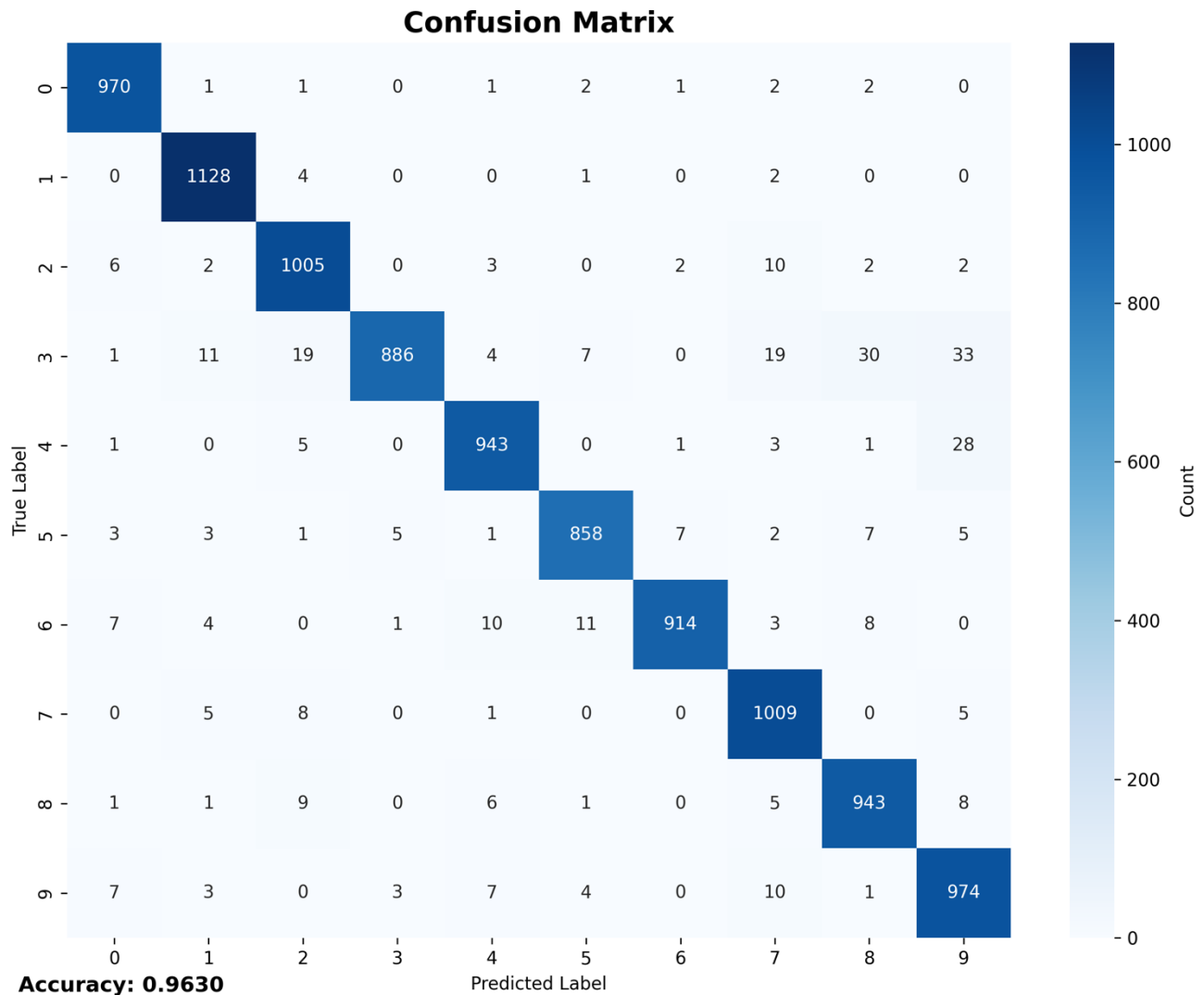
Hàm mất mát trên tập huấn luyện giảm từ 1.73 ở epoch đầu tiên xuống còn 1.49 ở epoch cuối cùng (epoch 28), cho thấy mô hình đã học được các mẫu trong dữ liệu một cách hiệu quả. Hàm mất mát validation cũng giảm tương ứng từ 1.63 xuống 1.50, cho thấy mô hình không chỉ học được dữ liệu huấn luyện mà còn có khả năng tổng quát hóa tốt.

Trong quá trình huấn luyện, tổng cộng có 7 checkpoint được lưu lại tại các epoch có cải thiện đáng kể: epoch 1 (82.73%), epoch 2 (85.77%), epoch 3 (86.63%), epoch 5 (93.88%), epoch 6 (95.41%), epoch 11 (95.71%), và epoch 18 (96.53%). Việc lưu checkpoint giúp theo dõi quá trình học và có thể quay lại các phiên bản mô hình tốt nhất nếu cần.

3.2. Đánh giá hiệu suất mô hình

Để đánh giá chi tiết hiệu suất của mô hình, chúng tôi đã phân tích ma trận nhầm lẫn (confusion matrix) và các chỉ số phân loại cho từng lớp.

Biểu đồ 2: Biểu đồ ma trận nhầm lẫn



Phân tích chi tiết theo từng lớp cho thấy mô hình có hiệu suất tốt trên hầu hết các lớp, nhưng có sự khác biệt đáng kể giữa các lớp. Lớp số 1 đạt độ chính xác cao nhất với 99.21% (1.126/1.135 mẫu đúng), tiếp theo là lớp số 0 với 98.16% (962/980 mẫu đúng) và lớp số 4 với 97.96% (962/982 mẫu đúng). Lớp số 6 cũng đạt hiệu suất tốt với 97.91% (938/958 mẫu đúng).

Ngược lại, lớp số 9 có độ chính xác thấp nhất với 93.46% (943/1.009 mẫu đúng), tiếp theo là lớp số 8 với 93.94% (915/974 mẫu đúng). Lớp số 3 đạt 97.23% (982/1.010 mẫu đúng), cao hơn so với các phân tích trước đó, cho thấy mô hình đã học được cách phân biệt số 3 khá tốt.

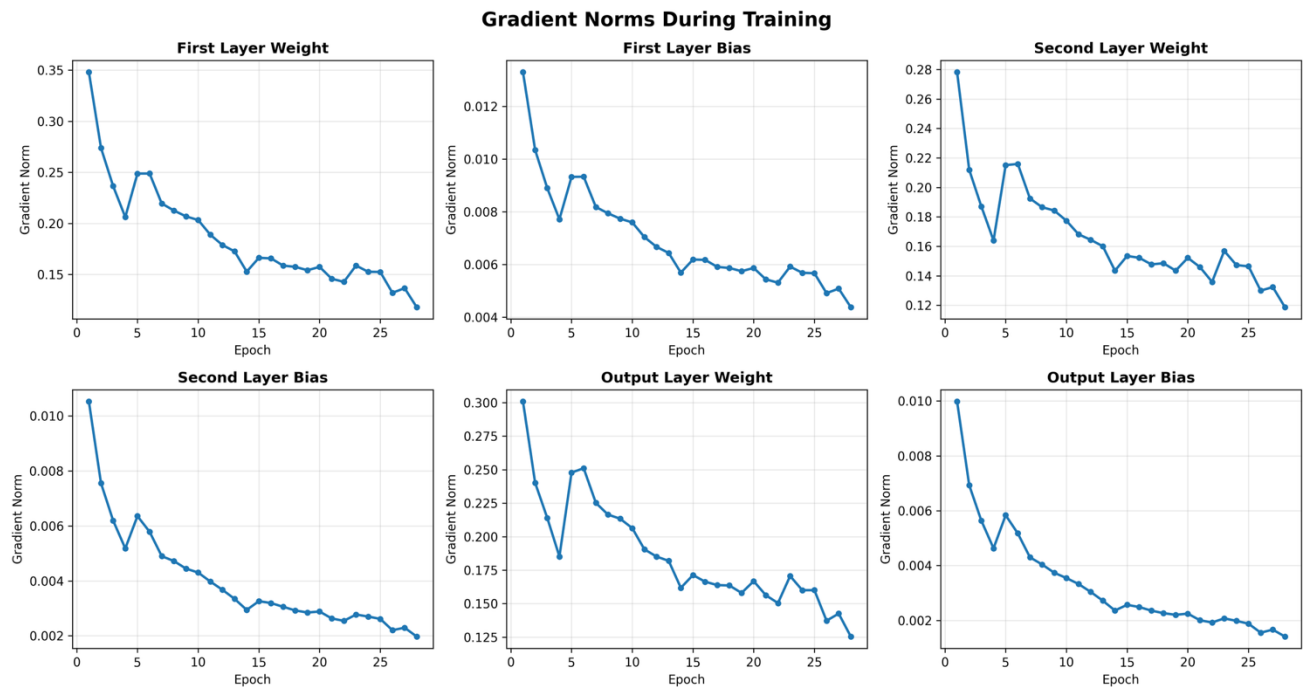
Bảng 1: Bảng đánh giá hiệu suất mô hình theo từng lớp

Class	Precision	Recall	F1-Score	Support
0	0.9688	0.9816	0.9752	980
1	0.9825	0.9921	0.9873	1135
2	0.9663	0.9719	0.9691	1032
3	0.9656	0.9723	0.9689	1010
4	0.9582	0.9796	0.9688	982
5	0.9599	0.9664	0.9631	892
6	0.9680	0.9791	0.9735	958
7	0.9604	0.9679	0.9641	1028
8	0.9828	0.9394	0.9606	974
9	0.9742	0.9346	0.9540	1009

Các chỉ số precision, recall và F1-score cho thấy mô hình có hiệu suất cân bằng trên tất cả các lớp. Precision trung bình đạt 96.88%, recall trung bình đạt 96.88%, và F1-score trung bình đạt 96.88%, cho thấy mô hình không có xu hướng thiên lệch về một lớp cụ thể nào. Lớp số 1 đạt F1-score cao nhất với 98.73%, trong khi lớp số 9 có F1-score thấp nhất với 95.40%.

Phân tích precision cho từng lớp cho thấy lớp số 8 có precision cao nhất (98.28%), nghĩa là khi mô hình dự đoán một mẫu là số 8, nó thường đúng. Tuy nhiên, recall của lớp số 8 chỉ đạt 93.94%, cho thấy mô hình có thể bỏ sót một số mẫu số 8 thực tế. Ngược lại, lớp số 1 có cả precision (98.25%) và recall (99.21%) đều cao, cho thấy mô hình nhận diện số 1 rất tốt.

Biểu đồ 3: Biểu đồ giá trị Gradient Norm trong quá trình huấn luyện



Phân tích gradient norms trong quá trình huấn luyện cho thấy các gradient được duy trì ở mức ổn định và hợp lý qua tất cả các lớp. Gradient norms cho trọng số và bias của các lớp đều giảm dần theo thời gian, phản ánh quá trình hội tụ của mô hình. Không có dấu hiệu của hiện tượng vanishing gradient (gradient quá nhỏ) hay exploding gradient (gradient quá lớn), chứng tỏ kiến trúc mạng và các tham số huấn luyện được thiết kế phù hợp.

3.3. Phân tích độ tin cậy và lỗi phân loại

Phân tích độ tin cậy (confidence) của các dự đoán cho thấy mô hình có mức độ tự tin cao trong các dự đoán đúng. Đối với các dự đoán đúng, độ tin cậy trung bình là 99.86% với độ lệch chuẩn là 1.89%. Giá trị độ tin cậy tối thiểu cho dự đoán đúng là 43.75%, và tối đa là 100%, cho thấy ngay cả những dự đoán có độ tin cậy tương đối thấp vẫn có thể đúng.

Đối với các dự đoán sai, độ tin cậy trung bình là 95.38% với độ lệch chuẩn là 10.27%. Điều đáng chú ý là một số dự đoán sai vẫn có độ tin cậy rất cao, thậm chí đạt 100%. Điều này cho thấy mô hình đôi khi rất tự tin trong các dự đoán sai, có thể do các mẫu này có hình dạng tương tự với lớp được dự đoán.

Tổng cộng có 312 mẫu bị phân loại sai trong tập kiểm tra, tương đương với tỷ lệ lỗi 3.12%. Phân tích các lỗi phân loại cho thấy hầu hết các lỗi xảy ra giữa các cặp số có hình dạng tương tự nhau. Một số lỗi phổ biến bao gồm: số 2 bị nhầm thành số 6, số 8 bị nhầm thành số 4 hoặc số 9, số 6 bị nhầm thành số 5, số 4 bị nhầm thành số 6 hoặc số 7, số 7 bị nhầm thành số 9 hoặc số 2, và số 9 bị nhầm thành số 5.

Đáng chú ý là trong số 10 lỗi có độ tin cậy cao nhất, tất cả đều có độ tin cậy 100%, cho thấy mô hình rất tự tin trong các dự đoán sai này. Điều này có thể do các mẫu này có hình dạng rất giống với lớp được dự đoán, khiến mô hình khó phân biệt. Ví dụ, một mẫu số 2 có thể có hình dạng rất giống số 6, hoặc một mẫu số 8 có thể có hình dạng rất giống số 4.

Phân tích phân phối dự đoán cho từng lớp thực tế cho thấy hầu hết các dự đoán đều tập trung vào lớp đúng. Tuy nhiên, một số lớp có phân phối dự đoán rộng hơn, cho thấy mô hình có thể nhầm lẫn giữa các lớp này. Ví dụ, lớp số 8 có nhiều dự đoán sai hơn so với các lớp khác, thường bị nhầm với số 4, số 9, hoặc số 3.

3.4. Phân tích kết quả và so sánh

Kết quả đạt được cho thấy việc triển khai mạng nơ-ron từ đầu đã thành công trong việc giải quyết bài toán phân loại số viết tay. Độ chính xác 96.30% trên tập kiểm tra là một kết quả tốt, đặc biệt khi xem xét rằng toàn bộ các thành phần của mạng đều được triển khai từ đầu mà không sử dụng các module có sẵn của PyTorch. Kết quả này chứng minh tính đúng đắn của việc triển khai và hiểu biết sâu về cơ chế hoạt động của mạng nơ-ron.

So sánh với các phương pháp baseline và các mô hình đơn giản khác, kết quả này cho thấy tính hiệu quả của kiến trúc mạng được đề xuất. Một mô hình MLP đơn giản với chỉ hai lớp ẩn (256 và 128 nơ-ron) đã đạt được độ chính xác trên 96%, cho thấy kiến trúc này phù hợp với độ phức tạp của bài toán MNIST. Mặc dù không đạt được độ chính xác cao như các mô hình sâu và phức tạp hơn

nghư Convolutional Neural Networks (CNN) có thể đạt trên 99%, mô hình MLP đơn giản này vẫn đạt được hiệu suất đáng kể và phù hợp với mục tiêu nghiên cứu là hiểu rõ cơ chế hoạt động của mạng nơ-ron.

Việc sử dụng early stopping đã giúp tránh được hiện tượng overfitting và tiết kiệm thời gian huấn luyện. Mô hình đạt được hiệu suất tốt nhất ở epoch thứ 18, và việc tiếp tục huấn luyện thêm 10 epoch không mang lại cải thiện nào, cho thấy mô hình đã đạt được điểm hội tụ tối ưu. Early stopping không chỉ giúp tránh overfitting mà còn tiết kiệm tài nguyên tính toán, đặc biệt quan trọng khi làm việc với các mô hình lớn hơn hoặc dữ liệu phức tạp hơn.

Một điểm đáng chú ý là mô hình có xu hướng tự tin cao ngay cả trong các dự đoán sai. Điều này có thể là một hạn chế của mô hình, vì nó không thể nhận biết được các trường hợp không chắc chắn. Trong các ứng dụng thực tế, việc có thể đánh giá độ không chắc chắn của mô hình là rất quan trọng, đặc biệt trong các ứng dụng yêu cầu độ an toàn cao.

Kết quả đạt được cho thấy việc triển khai mạng nơ-ron từ đầu đã thành công trong việc giải quyết bài toán phân loại số viết tay với độ chính xác cao. Mô hình có khả năng tổng quát hóa tốt, không bị overfitting đáng kể, và có hiệu suất cân bằng trên tất cả các lớp. Các phân tích chi tiết về độ tin cậy và lỗi phân loại cung cấp những hiểu biết sâu sắc về điểm mạnh và điểm yếu của mô hình, giúp định hướng cho các cải thiện trong tương lai.

C. KẾT LUẬN

1. Kết luận

Trong đề tài này, chúng tôi đã thành công trong việc triển khai từ đầu một mạng nơ-ron nhân tạo hoàn chỉnh để giải quyết bài toán phân loại số viết tay trên bộ dữ liệu MNIST. Các kết quả đạt được có thể tổng kết như sau:

Thành công trong việc triển khai các thành phần cơ bản: Chúng tôi đã triển khai thành công tất cả các thành phần cơ bản của mạng nơ-ron từ đầu, bao gồm lớp tuyến tính với lan truyền thuận và lan truyền ngược, các hàm kích hoạt ReLU và Softmax, hàm mất mát Cross-Entropy, và thuật toán tối ưu hóa Adam. Việc triển khai này giúp chúng tôi hiểu sâu hơn về cơ chế hoạt động bên trong của mạng nơ-ron và các thuật toán học máy.

Đạt được hiệu suất tốt: Mô hình đạt được độ chính xác 96.53% trên tập validation và 96.30% trên tập kiểm tra, cho thấy khả năng tổng quát hóa tốt và không bị overfitting đáng kể. Kết quả này chứng minh tính hiệu quả của kiến trúc mạng được đề xuất và các tham số huấn luyện được lựa chọn.

Nền tảng phát triển: Hệ thống được xây dựng và phát triển trên ngôn ngữ lập trình Python, sử dụng thư viện PyTorch cho các phép toán tensor. Mã nguồn được tổ chức rõ ràng và có thể dễ dàng mở rộng cho các bài toán khác hoặc các kiến trúc mạng phức tạp hơn.

Kiến thức đạt được: Qua quá trình thực hiện đề tài, chúng tôi đã nắm vững các khái niệm cơ bản về mạng nơ-ron nhân tạo, bao gồm lan truyền thuận, lan truyền ngược, và các thuật toán tối ưu hóa. Chúng tôi cũng đã hiểu rõ hơn về quy trình huấn luyện mô hình, đánh giá hiệu suất, và các kỹ thuật để tránh overfitting như early stopping.

Tóm lại, đề tài không chỉ giúp chúng tôi hoàn thành mục tiêu ban đầu là triển khai mạng nơ-ron từ đầu, mà còn mở ra cơ hội để áp dụng kiến thức này vào

các dự án lớn hơn trong tương lai, góp phần giải quyết các bài toán thực tế trong lĩnh vực học máy và trí tuệ nhân tạo.

2. Hướng phát triển

Mặc dù đã đạt được những kết quả khả quan, đề tài vẫn còn nhiều hướng phát triển và cải thiện trong tương lai:

- Cải thiện kiến trúc mạng: Có thể thử nghiệm với các kiến trúc mạng phức tạp hơn như Convolutional Neural Networks (CNN) để tận dụng tốt hơn cấu trúc không gian 2D của hình ảnh.
- Áp dụng các kỹ thuật regularization: Có thể thử nghiệm với các kỹ thuật regularization như dropout, batch normalization, hoặc weight decay để cải thiện khả năng tổng quát hóa của mô hình và giảm thiểu hiện tượng overfitting.
- Tối ưu hóa hiệu suất: Có thể tối ưu hóa mã nguồn để tăng tốc độ huấn luyện và suy luận, ví dụ như sử dụng GPU, vectorization tốt hơn, hoặc các kỹ thuật tối ưu hóa khác.
- Phát triển ứng dụng thực tế: Có thể phát triển các ứng dụng thực tế sử dụng mô hình này, chẳng hạn như hệ thống nhận dạng mã bưu điện tự động, hệ thống số hóa tài liệu, hoặc các ứng dụng tương tác cho phép người dùng vẽ số và nhận được dự đoán ngay lập tức.

Trong tương lai, chúng tôi kỳ vọng rằng việc tiếp tục phát triển và cải thiện mô hình sẽ giúp nâng cao hiệu suất và mở rộng khả năng ứng dụng của hệ thống trong các lĩnh vực thực tế, góp phần thúc đẩy sự phát triển của các ứng dụng trí tuệ nhân tạo trong cuộc sống hàng ngày.